

EV Charge DZ

Upgrade, Fix and KPI Measurement Report

Admin Dashboard • Backend API • MySQL Database

Report date: May 1, 2026
Project root: C:/Users/FODHIL/OneDrive/Desktop/MFE
Scope: Current workspace analysis plus previous fix report evidence
Verification: Backend build, frontend production build, code and schema metrics

Contents

1	Executive Summary	1
2	Project Analysis	2
2.1	Architecture	2
2.2	Main Functional Coverage	2
3	Upgrade and Fix Register	4
4	KPI Measurements	7
4.1	Build and Verification KPIs	7
4.2	Codebase Size KPIs	8
4.3	Security and Operations KPIs	8
5	Resource Evidence	9
5.1	Local Sources Used	9
5.2	Research Resources Present in Workspace	10
6	Residual Risks and Recommendations	11
7	Conclusion	13

Chapter 1

Executive Summary

This report consolidates the upgrades, corrective fixes, KPI measurements, and resource evidence for the EV Charge DZ platform. The project is a multi-tenant electric vehicle charging operations system composed of a React/Vite dashboard, a Node.js/Express API, and a MySQL database structured around operational views.

The current workspace confirms that the platform was upgraded from a mostly demo-oriented dashboard toward a backend-connected operational MVP. The most important changes are real authentication, server-side OTP verification, role-based access, validated API writes, MySQL-backed KPI queries, a normalized database schema, demo seeding scripts, structured server logging, safer station deletion, and production build readiness.

Important limitation: the workspace is not a Git repository, so exact chronological history could not be reconstructed from commits. The upgrade history below is derived from the current source code, the previous LaTeX fix report, package configuration, database schema, and build outputs.

Chapter 2

Project Analysis

2.1 Architecture

Layer	Current implementation
Frontend	React 18, Vite 6, TypeScript, React Router, Tailwind, MUI/Radix UI components, Recharts, Leaflet map integration.
Backend	Express 4 API in TypeScript, MySQL connection pool, JWT authentication, bcrypt password hashing, Zod validation, Winston logging.
Database	MySQL schema with normalized tables and view-model views used by dashboard and mobile-facing read models.
Security	JWT bearer auth, role guards, rate limits on login and OTP request, server-side OTP state, environment-driven secrets.
Operations	Demo seed scripts for tenants, staff, app users, stations, connectors, sessions, billing, tickets, and alerts.

2.2 Main Functional Coverage

- Multi-tenant operation for Sonelgaz and SAEIG.
- Three dashboard roles: `super_admin`, `tenant_admin`, and `technician`.
- Dashboard KPIs: active stations, active sessions, total energy, revenue, faults, offline stations, and uptime rate.
- Station management: list, detail, create, update, connector summary, status filtering, and soft delete.
- Session monitoring: tenant/station/status filters and paginated results.
- Ticket management: create, assign/update, activity timeline, priority and status filters.
- Billing and alert read models from database views.

- Frontend fallback to mock data when no backend URL is configured.

Chapter 3

Upgrade and Fix Register

#	Upgrade or fix	Evidence in project	KPI / impact
1	Backend API integration	<code>server/src/index.ts</code> , <code>server/src/routes/*.ts</code> , <code>project/.../services/api.ts</code> . The frontend now has a centralized API client and the server exposes REST resources for auth, tenants, users, stations, connectors, sessions, tickets, billing, alerts, and dashboard KPIs.	28 route handlers measured across backend route files.
2	MySQL operational schema	<code>database/ev_charge_dz.sql</code> . The schema defines tenants, users, privileges, app users, stations, connectors, sessions, tickets, ticket activities, billing, and alerts.	12 tables, 8 views, 480 SQL lines.
3	Dashboard KPI upgrade	<code>server/src/routes/dashboard.ts</code> . KPIs are computed from stations and charging sessions with tenant scoping.	Backend returns active stations, active sessions, energy, revenue, faults, offline count, and uptime rate.
4	Authentication hardening	<code>server/src/routes/auth.ts</code> , <code>project/.../pages/Login.tsx</code> . Login uses bcrypt password comparison and JWT issue; frontend stores the received token and calls real backend login.	Password hashing uses bcrypt cost 12 in seed scripts; backend TypeScript build passes.

#	Upgrade or fix	Evidence in project	KPI / impact
5	Server-side OTP verification	<code>server/src/routes/auth.ts</code> , <code>project/.../services/otpService.ts</code> , <code>project/.../pages/OtpVerify.tsx</code> . OTP generation, expiry, and attempt tracking moved server-side.	OTP TTL is 5 minutes; max attempts is 3.
6	Authentication rate limiting	<code>server/src/routes/auth.ts</code> . Login and OTP request endpoints use <code>express-rate-limit</code> .	Login limit: 10 attempts / 15 minutes. OTP request limit: 5 requests / 5 minutes.
7	Role-based routing	<code>project/.../routes.tsx</code> , <code>project/.../components/auth/RequireAuth.tsx</code> . Routes are restricted by role and technicians are redirected to their task page.	3 role profiles enforced in frontend routing.
8	Backend input validation	<code>server/src/routes/stations.ts</code> , <code>server/src/routes/users.ts</code> , <code>server/src/routes/tickets.ts</code> . Zod schemas validate create/update payloads.	6 Zod object schemas measured. Invalid requests return HTTP 400 before database writes.
9	Pagination on list endpoints	<code>server/src/routes/stations.ts</code> , <code>server/src/routes/users.ts</code> , <code>server/src/routes/tickets.ts</code> , <code>server/src/routes/sessions.ts</code> . List queries use <code>page/limit</code> or <code>limit/offset</code> .	Default limit 50; maximum 200. Reduces oversized result sets.
10	Structured logging	<code>server/src/logger.ts</code> , <code>server/src/index.ts</code> . Winston logger with timestamp, level, message, and stack support is wired into startup and global error handling.	Backend errors are centrally logged instead of only returning generic 500 responses.
11	Safer station deletion	<code>server/src/routes/stations.ts</code> , <code>database/ev_charge_dz.sql</code> . Station delete sets <code>deleted_at</code> ; station overview view excludes deleted stations.	Preserves station history and foreign-key relationships.

#	Upgrade or fix	Evidence in project	KPI / impact
12	Frontend error boundary	<code>project/.../components/ErrorBoundary.tsx</code> , <code>project/.../components/layout/DashboardLayout.tsx</code> . Dashboard content is wrapped in an error boundary.	Route-level UI crashes show a recoverable fallback instead of blanking the entire dashboard.
13	Demo data seeding	<code>server/scripts/seed-admin.ts</code> , <code>server/scripts/seed-demo-data.ts</code> . Scripts seed tenants, staff, privileges, app users, stations, connectors, sessions, billing, tickets, and alerts.	Demo dataset includes 2 tenants, 5 dashboard staff including super admin, 4 mobile app users, 8 stations, 11 connectors, 10 sessions, 8 billing records, 4 tickets, and 4 alerts.
14	Production build readiness	<code>server/package.json</code> , <code>project/.../package.json</code> . Both backend and frontend have production build scripts.	Backend build passed with <code>tsc</code> ; frontend Vite build passed with 3107 transformed modules.

Chapter 4

KPI Measurements

4.1 Build and Verification KPIs

Measurement		Result	Evidence
Backend build	TypeScript	Verified	<code>npm run build</code> in <code>server</code> ; command completed with exit code 0.
Frontend build	production	Verified with warning	<code>npm run build</code> in frontend project completed with exit code 0 after allowing Vite/esbuild spawn.
Frontend transformed modules		3107	Vite build output.
Frontend build time		10.15 seconds	Vite build output.
Frontend JS bundle		1322.68 kB, gzip 385.85 kB	Vite output for <code>assets/index-*.js</code> .
Frontend CSS bundle		154.10 kB, gzip 32.66 kB	Vite output for <code>assets/index-*.css</code> .
Bundle size warning		Warning	Vite warns that one chunk is larger than 500 kB after minification.

4.2 Codebase Size KPIs

Measurement	Value	Scope
Backend TypeScript files	14	server/src.
Backend TypeScript lines	872	server/src.
Frontend TS/TSX/CSS files	97	project/AdminDashboardforEVOperations/src.
Frontend TS/TSX/CSS lines	12835	project/AdminDashboardforEVOperations/src.
API route handlers	28	server/src/routes.
Database tables	12	database/ev_charge_dz.sql.
Database views	8	database/ev_charge_dz.sql.
Database schema lines	480	database/ev_charge_dz.sql.

4.3 Security and Operations KPIs

KPI	Current value	Meaning
Authentication roles	3	Super admin, tenant admin, technician.
Login rate limit	10 / 15 minutes	Reduces brute-force login attempts.
OTP request limit	5 / 5 minutes	Reduces OTP abuse in demo or production-like mode.
OTP expiry	5 minutes	Limits validity window for second-factor codes.
OTP attempts	3	Locks verification after repeated failures.
Database connection limit	10	MySQL pool concurrency setting.
List endpoint default page size	50	Controls baseline response size.
List endpoint max page size	200	Prevents very large list responses.

Chapter 5

Resource Evidence

5.1 Local Sources Used

Resource	Information taken from it
EV_Charge_DZ_Fix_Report.tex	Previous fix inventory: hardcoded super admin code, OTP, JWT secret, rate limiting, validation, error boundary, pagination, technician redirect, API error handling, logging, and soft deletes.
README.md	Product context, user roles, questionnaire themes, OCPP/smart charging direction, and Algeria charging-platform needs.
project/AdminDashboard forEVOperations/USAGE_GUIDE.md	User-facing workflows, dashboard modules, status meanings, tenant behavior, and common operational scenarios.
server/src/index.ts	API composition, CORS configuration, health endpoint, route registration, and global error handling.
server/src/routes/*.ts	Authentication, RBAC, endpoint behavior, rate limits, validation, pagination, data mapping, and KPI logic.
server/src/db.ts	MySQL pool configuration and connection limit.
server/src/logger.ts	Structured Winston logging format.
database/ev_charge_dz.sql	Data model, relationships, indexes, views, seed tenants, and privilege catalogue.
server/scripts/seed-admin.ts and server/scripts/seed-demo-data.ts	Demo accounts, privileges, tenants, stations, connectors, sessions, billing, tickets, and alerts.
project/AdminDashboard forEVOperations/src/app/**/*.*tsx	Frontend routing, pages, auth guard, error boundary, dashboard views, data fallback, and service integration.
package.json, server/package.json, frontend package.json	Build scripts, dependencies, and technology stack.

5.2 Research Resources Present in Workspace

The workspace also contains research and project documents that provide broader EV charging and smart-charging context. They were inventoried as project resources, but KPI measurements in this report were derived from source code and build outputs.

- OCPP_Recherche.pdf
- SmartCharging+scAI.pdf
- ComprehensiveAnalysisofElectricVehicleChargingInfrastructure_OCOPProtocolEvolutionandSmartCharging.pdf
- OCOPProtocolEvolutionandSmartCharging_FR.pdf
- OCOPProtocolEvolutionandSmartCharging_FR_V2.pdf
- ProjectIFAG.pdf
- MFEFodhil&Chibani.pdf
- Rapport_Comparatif_EV_Admin_FR.pdf

Chapter 6

Residual Risks and Recommendations

Risk	Evidence	Recommendation
Frontend bundle size	Vite reports a JS chunk of 1322.68 kB and warns about chunks over 500 kB.	Add route-level dynamic imports and manual chunks for large chart/map/UI dependencies.
Enum mismatch: station status	Frontend and database include charging/fault ; station update schema accepts occupied and omits fault .	Align Zod schemas, TypeScript types, and MySQL enums.
Enum mismatch: ticket status	Ticket update schema accepts closed ; database and frontend use escalated instead.	Standardize allowed ticket statuses before production use.
Enum mismatch: user status	User update schema accepts inactive ; database uses suspended .	Replace inactive with suspended or migrate database enum intentionally.
No automated test suite	Root package has no test script; verification currently relies on builds and code review.	Add API integration tests and frontend smoke tests for login, role routing, dashboard KPI load, and CRUD actions.
Demo OTP returns code to UI	Server returns demoCode for presentation convenience.	Remove demoCode and send OTP through email/SMS before production.
JWT in localStorage	Frontend stores bearer token in localStorage.	For production, evaluate secure HttpOnly cookies or stricter XSS controls.

Risk		Evidence	Recommendation
Environment	validation	Backend expects <code>JWT_SECRET</code> with non-null assertion.	Add startup validation for required environment variables.

Chapter 7

Conclusion

The project now presents a coherent operational MVP for EV charging administration. The strongest upgrades are the move to real backend authentication, MySQL-backed operational data, role-aware routing, KPI computation, validation, rate limiting, structured logging, and safer data lifecycle handling. The builds confirm that the backend compiles and the frontend produces a production bundle.

Before final production deployment, the highest-value next step is to resolve the status enum mismatches and add automated tests. The frontend bundle-size warning should also be treated as a performance optimization target, especially because map, chart, and UI libraries are included in the current application bundle.